

From Felicity to Protocol

How Speech-Act Theory Became Executable Social Order

Watson Hartsoe

Working Paper — AI-augmented research process

18 August 2026

Evidence, prompts, and making history preserved with the publication.

Abstract

Speech-act theory is often invoked as an intellectual ancestor of workflow systems, conversational agents, and institutional software. The genealogy is usually told too quickly: philosophical distinctions become message labels, message labels become finite states, and social action becomes code. This paper reconstructs a more difficult lineage from J. L. Austin through Winograd and Flores’s language/action perspective, the Coordinator and ActionWorkflow systems, agent-oriented programming, FIPA communicative-act standards, and commitment-based multiagent protocols. I argue that the decisive change is not the progressive removal of context but the *migration of semantic burden*. Austin locates performative success across conventional procedure, appropriate persons, circumstances, uptake, and sincerity. Winograd and Flores retain this distributed semantics through hermeneutic background while using commitments to organize action. Workflow systems then make a subset of that structure executable as roles, permitted moves, and persistent incompletions. A parallel artificial-agent lineage first relocates meaning into formal beliefs and intentions, then confronts the problem that such states are private and therefore poor grounds for interoperable standards. Singh’s public-semantics critique and Yolum and Singh’s commitment protocols move meaning again, into publicly attributable normative relations. This history reveals a recurring design problem: executable social semantics never eliminates background; it chooses where background will reside and which parts will be testable. The paper closes by distinguishing event, normative, interpretive, and contestability semantics as separate requirements for institutional programming.

Keywords: speech acts; language/action perspective; workflow; agent communication languages; social commitments; formal semantics; institutional computation; CSCW

1 Introduction

A promise is an unusually difficult object for computation. Its words are short. Its consequences are not. Whether a promise succeeds depends on who speaks, under what circumstances, through what convention, to whom, with what uptake, and against which background of expectations. After the utterance disappears, an obligation can remain. The social fact is therefore neither identical to

the string nor exhausted by a momentary internal intention. It is a distributed event whose effects persist in a normative relation.

During the last half century, researchers repeatedly tried to make such relations computationally explicit. The results range from office-workflow systems to formal languages for autonomous software agents. What makes this lineage unusual is that the path from philosophy to execution can be followed with enough technical specificity to see where conceptual commitments entered software. Requests and promises became typed moves. Conversational positions became finite states. Outstanding obligations became persistent tokens. Later, beliefs and intentions became formal agent state, communicative acts acquired feasibility preconditions and rational effects, and social commitments became fluents that could be created, discharged, cancelled, released, assigned, and delegated. [1, 3, 11, 13, 15]

The tempting story is one of increasing formalization: context-rich social theory is progressively sharpened until it becomes executable. That story misses the most important transformation. Formalization does not simply reduce context. It *moves* context. A condition that was once located in conventional circumstance may reappear as a role predicate, a database field, a private belief formula, a public commitment, or an assumption about compliant behavior. Each move makes some questions easier to compute and others harder to ask.

I call this the *migration of semantic burden*. The phrase names a historical pattern, not a term used by the authors considered here. At each stage, a system must answer a practical question: where will the conditions that make an action meaningful live? Austin disperses them across procedure, actor, circumstance, intention, and subsequent conduct. Winograd and Flores insist that listening occurs in an irreducible background while making commitment central to organizational coordination. The Coordinator and ActionWorkflow make selected elements of that coordination operational. Agent-oriented programming then places semantic burden in explicitly represented mental states. FIPA standardizes that strategy. Singh objects that a normative standard cannot depend on private mental states that other autonomous agents cannot inspect, and commitment protocols relocate meaning into public social relations. [1, 3, 6, 7, 14, 15]

This reconstruction yields two claims. First, there is no single straight line from speech-act theory to protocol code. There are at least two partially independent operational lineages: one through human organizational workflow and CSCW, another through artificial-agent communication. They share concepts and sometimes cite common speech-act sources, but resemblance must not be mistaken for direct influence. Second, the two branches encounter structurally similar failures. Lucy Suchman argues that human communicative force cannot always be cleanly isolated prior to interaction; Munindar Singh argues that an agent communication standard cannot ground compliance in inaccessible private intention. These are not the same critique, and there is no claim here that one caused the other. Their collision nevertheless exposes a common design problem: a formal semantics fails when the information required to determine meaning is unavailable to the participants or verifier at the point where the formalism demands it. [8, 14]

The consequence is constructive. A more adequate executable social semantics must distinguish at least four layers: *event semantics* (what happened and in what causal order), *normative semantics* (what public commitments, permissions, or liabilities now hold), *interpretive semantics* (what the act and its conditions of satisfaction mean in context), and *contestability semantics* (who may challenge, revise, waive, or refuse the categories and commitments themselves). Existing systems are strongest in different layers. Confusing those strengths produces both technical and political errors.

2 Felicity before formal semantics

Austin’s account begins from a fact that later protocol languages repeatedly rediscover: some utterances do things. Yet the doing is not contained in the words alone. The marriage formula, the naming of a ship, a bet, a promise, or an official pronouncement works only under appropriate conditions. Austin’s early felicity schema requires an accepted conventional procedure with a conventional effect; appropriate persons and circumstances; correct and complete execution; and, for some acts, the requisite thoughts, feelings, intentions, and subsequent conduct. [1]

This architecture already resists a message-centered semantics. The same surface form can succeed or fail because the speaker lacks standing. A judge’s pronouncement and an ordinary person’s identical sentence do not necessarily have the same institutional effect. A ship-naming formula spoken by someone who was not appointed to name the vessel misfires. A bet generally requires uptake by another party. Austin’s examples therefore make authority, procedure, and relation part of what it is for the act to occur at all. [1]

The promise makes the temporal structure especially clear. Austin separates sincerity from conventional consequence. An insincere promise can be defective without becoming an ordinary false statement. More importantly for later computational work, promising changes what subsequent conduct counts as. It places the promisor under an obligation; later action can be assessed as fulfillment or violation because the normative situation has changed. The event is brief, but the relation persists. [1]

This is a crucial stepping stone toward executable institutions. It is possible to read a promise as a state transition:

no obligation $\rightarrow$ successful promise $\rightarrow$ obligation in force.

But that reconstruction must not be projected backward as Austin’s own formalism. His point is precisely that the transition is conditioned by conventions, standing, uptake, and context. The useful computational insight is not that Austin secretly wrote a state machine. It is that a performative has both an *event* and a *normative afterlife*. Later systems can choose to formalize one, the other, or the relation between them.

Austin’s discussion of explicit performatives adds a second bridge. “I shall be there” may or may not be heard as a promise; “I promise that I shall be there” makes the force more explicit. Explicitness reduces one ambiguity by naming the kind of act being performed. It does not eliminate the felicity conditions that make the act successful. This distinction becomes easy to lose once a software interface offers PROMISE as a menu item. The explicit label then appears not merely to clarify the act but to define it.

A first design principle follows. **Act typing and act validity are different semantic problems.** A formal language can know that a user selected PROMISE while remaining uncertain whether the promise was authorized, understood, accepted, sincere, feasible, or satisfied. Later standards often make this distinction technically precise as preconditions and effects. The lineage begins, however, with a warning: even an explicit performative does not carry all of its own semantics.

3 Winograd and Flores: speech acts transformed by background

The move from ordinary-language philosophy to organizational computing did not occur through a simple transfer of Austin or Searle into software. *Understanding Computers and Cognition* explicitly constructs a broader philosophical synthesis. Winograd and Flores identify Heidegger and Gadamer as formative influences on their own orientation, then present speech-act theory as a starting point rather than a complete account. Their language/action perspective joins communicative acts to background, listening, interpretation, commitment, and breakdown. [3]

This matters for genealogy because later summaries often remember the speech-act vocabulary and forget the hermeneutic limit built into the same theory. In the book's discussion of "listening in a background," an utterance does not arrive with a self-sufficient semantic type. It is heard against historically formed practices and expectations. The listener is not a passive decoder. What becomes intelligible as a request, promise, declaration, or assessment depends on a background that is not wholly represented in the message. [3]

At the same time, Winograd and Flores make commitment the practical center of organizational language. A request initiates a possible commitment. A promise transforms the relation between requester and performer. A report of completion does not itself settle the matter; the customer assesses whether the conditions of satisfaction have been met. The familiar conversation-for-action pattern is therefore not merely a taxonomy of utterance types. It is a representation of how public commitments are opened, negotiated, performed, and closed. [3]

The conversation-for-action diagram has a syntax-like quality. From a request, a promise, counteroffer, or refusal may make sense; after performance, a report and an assessment may follow. It is possible to draw these possibilities as states and arrows. But the book itself prevents an easy identification of the diagram with a complete finite-state ontology. Conditions of satisfaction are not objective facts independent of participants' interpretation. The customer and performer can disagree about whether the work is done. The practical labor that satisfies the request is also not identical to the sequence of speech acts that coordinates it. [3]

These distinctions create a three-part structure before any software is considered. There is a *conversational event* (request, promise, report, assessment); a *normative relation* (what each party is now committed to); and a *practical world* in which work is or is not performed. Interpretation connects them. A system that stores only the conversational events can lose the commitment. A system that stores only the commitment can lose the contested meaning of its conditions. A system that treats a completion report as equivalent to material completion can lose the world.

The book's anti-objectivist argument makes the tension explicit. Winograd and Flores reject the project of a complete, background-free language for human understanding. Their design project is therefore not best understood as an attempt to capture all of conversation in formal states. It is a more limited wager: certain recurrent structures of commitment can be made explicit and used to support coordination even though understanding remains situated. [3]

This qualification becomes crucial once the ideas become software. A philosophical diagram can remain knowingly incomplete. An interface must decide what a user can click.

4 The Coordinator: when an ontology becomes an action space

The Coordinator and ActionWorkflow make the philosophical lineage technically consequential. Here requests, promises, completions, and satisfactions cease to be only analytic categories. They become data structures and transition conditions. The 1992 ActionWorkflow paper describes a workflow language interpreter, a transaction database, workflow processors, and agents that calculate workflow states and next actions. The Coordinator patent specifies finite-state machinery, participant roles, permitted moves, and persistent records of incompletions. [6, 7]

This is a stronger claim than saying that software was “inspired by” speech-act theory. The categories acquire machine consequence. If a user’s current role and conversational state are inputs to the routine that assembles available moves, the state model determines what the interface presents as actionable. A formal ontology returns to the social world as an affordance structure.

The incompleteness mechanism is particularly revealing. An unfinished obligation can be represented by a token that remains active until a recognized move changes its status. Something absent - the response or fulfillment that has not yet occurred - becomes a durable computational object. The system thereby acquires organizational memory. It can display, route, and perhaps escalate what remains outstanding. [7]

The gain is real. Ordinary email can leave obligations diffuse and difficult to recover. A commitment-aware system can make pending work legible. Yet legibility is not neutral. To represent an incompleteness, the system must decide what counts as an obligation, who owes it, when it is discharged, and which events can alter it. A social relation becomes a state object with an owner, lifecycle, and recognized transitions.

The patent’s handling of distributed message order adds another layer. Network transport may deliver messages out of sequence, so the system can reorder them before applying finite-state transitions. This is sensible when the system is recovering a prior causal order obscured by latency. It becomes more difficult when interaction itself is concurrent or when the meaning of one event depends retrospectively on another. A deterministic conversational ledger requires more than timestamps; it requires a theory of which order is socially authoritative.

ActionWorkflow’s own account keeps the hidden dependency visible. Agreement rests partly on a shared background of assumptions and standard practices. The machine may know that a workflow is in AGREEMENT while the participants disagree about what “acceptable,” “urgent,” or “complete” means. The formal state is therefore operationally precise and semantically underdetermined. [6]

Suchman’s critique targets exactly this crossing from category to infrastructure. Drawing on ethnomethodology and conversation analysis, she argues that communicative action is irreducibly interactional and circumstantially contingent; illocutionary force is not always a pre-existing object that can simply be selected before interaction. More broadly, she asks what happens when analytic categories become ordering devices through which organizational conduct is monitored and made accountable. [8]

The important issue is not whether categories are ever useful. Winograd’s later response explicitly defends formal structures as partial instruments rather than complete descriptions. The unresolved question is deployment: who defines the categories, whether use is optional, how easily categories can be revised, what sanctions attach to them, and whether actions outside the ontology remain possible. [9, 10]

This is the first major transformation in the lineage. **Felicity conditions that were dis-**

tributed across social practice become selectively compiled into roles, states, tokens, and permitted moves. What cannot be compiled does not vanish; it becomes background.

5 A parallel branch: speech acts become agent-programming primitives

A second lineage begins from a different technical problem. Artificial agents need to coordinate with other autonomous programs. In object-oriented programming, messages can be method invocations with implementation-specific semantics. In agent-oriented programming, Yoav Shoham proposed a higher-level abstraction in which agents have formally represented mental states and communicate with typed acts such as informing, requesting, offering, promising, and declining. [11]

Here the philosophical vocabulary becomes executable in a more literal sense. Beliefs, choices, capabilities, commitments, and obligations are parts of program state. Communication is not an unconstrained byte string; its type participates in the interpretation of what the receiving agent should infer or do. Shoham's AGENT0 work thus belongs at a deeper transfer level than metaphor. It couples a formal language, interpreter, mental-state representation, and speech-act-like message primitives. [11]

Yet Shoham is more cautious than a simple “software has beliefs” reading suggests. The relation between formal categories and commonsense concepts is approximate. Different applications may warrant different categories or interpreters. That warning is important because agent communication research soon faced pressure for standardization. Interoperability requires independently built agents to agree not only on syntax but on what communicative acts mean.

Smith and Cohen's 1996 work develops one response. Instead of treating finite conversational patterns as primitive, they ground communicative acts in a formal theory of joint intentions and teamwork. Dialogue structures can then emerge from logical relationships among the acts. The conversation-for-action pattern appears in this literature not simply as a fixed workflow to be copied, but as something that can be derived from deeper semantics. [12]

This reverses the Coordinator architecture. In a sequence-first system, the finite-state structure defines which moves are legal. In a semantics-first system, the meaning of acts constrains which sequences are coherent, and an FSM can be a projection of that deeper model. The two systems may generate the same happy path while diverging dramatically under refusal, cancellation, unexpected opportunity, or concurrent action.

The apparent gain in semantic depth introduces a new dependency. Mentalistic semantics refers to beliefs, intentions, and other internal attitudes. The model can be mathematically precise while the relevant facts remain private to the agent implementation. Assumptions of good-faith communication help bridge the gap between private state and observable behavior, but those assumptions become part of the semantic machinery. [12, 11]

Formalization has therefore moved the background again. In the organizational workflow branch, unformalized context sits outside the transition table. In the mentalistic agent branch, much of the semantic burden is placed inside the agent as formal internal state. What counts as context has changed location, not disappeared.

6 FIPA and the standardization of mentalistic meaning

The FIPA Communicative Act Library turns this research program into an interoperability standard. It provides a shared catalogue of communicative acts - REQUEST, INFORM, REFUSE, PROPOSE, AGREE, CONFIRM, and many others - and specifies their semantics through formal feasibility preconditions and rational effects. FIPA also supplies structured interaction protocols composed from communicative acts. [13]

The resemblance to Austin's felicity architecture is striking but must be handled carefully. Austin asks under what conditions an act is happy; FIPA asks which formal conditions make an act feasible and which effects a rational agent seeks. Both place constraints around an explicitly typed act. Their ontologies are different. Austin's conditions include accepted conventions, appropriate persons, circumstances, sincerity, and subsequent conduct. FIPA's formulas are built from representations of agent attitudes and actions. Structural resemblance is not proof that FIPA simply formalized Austin's rules.

The more consequential point is where FIPA places semantic burden. A public message is interpreted through formulas about beliefs, intentions, uncertainty, feasibility, and rational effects. This makes the standard formally articulate. It also creates a conformance problem. If one independently implemented agent sends INFORM, how can another system verify the sender's relevant belief state? If REQUEST is defined through private mental conditions, what can a public compliance test actually observe? [13]

The FIPA interaction-protocol layer adds a second kind of closure. Protocols are described as pre-enumerated inter-agent plans. They can contain alternatives and branches, so "pre-enumerated" should not be confused with a simple linear script. Still, legal interaction is represented substantially through expected message trajectories. This approach supports analyzability and standard behavior, but it can constrain autonomous agents when an unanticipated action would satisfy the same social obligations.

The agent branch has now reproduced two problems familiar from human workflow. First, a finite set of act types defines a machine-readable social vocabulary. Second, protocol structure constrains available continuations. The difference is that the agent literature has a formal semantic layer beneath those sequences. That semantic layer, however, is private.

7 Singh: from private minds to public social semantics

Munindar Singh's 1998 critique changes the criterion for success. A communication standard, he argues, needs a semantics that supports public compliance testing among autonomous heterogeneous agents. A standard whose meaning depends on one agent reading another's private beliefs or intentions is ill suited to that task. Communication is public; normative semantics should therefore emphasize social agency and public context. [14]

The shift is easy to underestimate. It is not merely a preference for simpler implementation. It changes what "meaning" is for. Mentalistic semantics asks what beliefs or intentions an act expresses or rationally changes. Public semantics asks what social relation is established such that independently controlled parties can determine whether interaction conforms to a standard.

Singh distinguishes personal from conventional meaning and private from public perspective. Private mental states can remain important to an agent's own decision process. They are simply

poor candidates for the normative substrate of interoperability. The public layer should instead rely on conventions and social context that can, in principle, be inspected by the parties. [14]

Here an unexpected crossing appears with Austin. Austin’s successful performative has a conventional effect that is not reducible to inner sincerity. Singh’s desired ACL has conventional public meaning that should not be grounded in private intention. The resemblance is analytically powerful, but there is not enough evidence here to make it a direct intellectual genealogy. The sources belong to different debates separated by decades and technical domains. The correct claim is narrower: both expose the instability of making private intention the sole ground of public normative consequence.

A second crossing appears with Suchman. Suchman argues that human communicative force can be interactionally achieved and cannot always be cleanly tagged in advance from an isolated speaker intention. Singh argues that an inter-agent standard cannot make compliance depend on internal states unavailable to other agents. These critiques are not equivalent. Suchman’s problem is situated meaning and interactional production; Singh’s is public testability under autonomy and heterogeneity. Yet both identify a failure mode in which the formal semantics demands information that is not publicly settled at the point of classification.

This independent convergence suggests a stronger design criterion: **the normative layer of an executable institution should depend only on predicates for which the institution has a legitimate public settlement procedure.** “Public” cannot simply mean “stored in the database.” It means that the relevant parties know how a fact is established, challenged, and revised.

That final clause matters because public semantics can itself become coercive. A ledger can be perfectly observable while encoding an unjust or disputed classification. Singh solves an epistemic problem in agent standards; he does not solve Suchman’s political problem of who gets to define the categories. The two critiques therefore meet only if public semantics is extended with contestability.

8 Commitments as executable public state

Commitment-based protocol research makes the public-semantics turn technically concrete. Yolum and Singh criticize sequence-oriented protocols such as finite-state and Petri-net specifications for overconstraining autonomous agents and for representing the order of actions more readily than their social meaning. Their alternative treats social commitments as the semantic substrate of protocol interaction. [15]

A commitment is a publicly attributable normative relation: one party is debtor, another creditor, and a condition specifies what is owed. The 2002 model gives this relation explicit operations. Commitments can be created, discharged, cancelled, released, assigned, and delegated. These changes are represented in an event-calculus framework through events that initiate or terminate commitment fluents. [15]

This is the most technically explicit point in the lineage considered here. The claim that a social concept has become executable no longer depends on analogy. A normative relation has a data representation, typed transformations, persistence conditions, and planning consequences. An agent can reason over the current commitment state and select among multiple trajectories that preserve or satisfy its obligations.

The move from sequence to commitment also changes the meaning of flexibility. A flexible protocol is not one without constraints. It relocates the invariant. Instead of specifying an exact

path, it specifies social conditions that valid paths must respect. New opportunities or exceptions can be handled at runtime if they preserve the required commitment semantics. [15]

This architecture addresses a problem visible in both the Coordinator and FIPA branches. It separates the meaning of an interaction from one canonical sequence of messages. The same normative outcome can be reached through different traces. Finite-state views can be compiled or derived for analysis rather than treated as the ontology itself.

But the move does not close the lineage. The commitment formalism still needs answers to Austinian and hermeneutic questions. Who was authorized to create or release the commitment? What if the debtor and creditor disagree about the condition? What evidence counts as discharge? What if the actor contests the protocol's standing to impose the relation? What if two events are concurrent and produce incompatible commitment updates? A public normative state can remain underdetermined by the event trace.

The semantic burden has moved again. What was once hidden in private belief is now represented publicly as a commitment. Background returns at the boundary conditions governing how commitments are created, interpreted, evidenced, and contested.

9 The migration of semantic burden

The lineage can now be stated more precisely. It is not a march from vagueness to rigor. It is a sequence of decisions about where to locate the conditions that make communicative action consequential.

1. **Austin: distributed felicity.** Semantic success is spread across conventional procedure, appropriate participants and circumstances, uptake, sincerity, and subsequent conduct. The words alone are insufficient. [1]
2. **Winograd and Flores: background plus commitment.** Speech-act distinctions are retained but embedded in listening, interpretation, history, and shared background. Commitment gives conversation an action-oriented structure without claiming a background-free semantics. [3]
3. **Coordinator/ActionWorkflow: selected conditions become executable state.** Roles, conversation positions, permitted moves, and incompletions are made explicit enough for a runtime to calculate next actions. Unformalized assumptions remain background dependencies. [6, 7]
4. **AOP and mentalistic ACLs: burden moves inward.** Meaning is grounded in formally represented beliefs, intentions, capabilities, commitments, and joint intentions. Precision increases, but normative interoperability now depends on private state and rationality assumptions. [11, 12, 13]
5. **Singh: burden moves outward.** A standard should use public social semantics because compliance among autonomous heterogeneous agents must be testable without mind-reading. [14]
6. **Commitment protocols: public normative relations become runtime state.** Meaning is represented through commitments and their operations; trajectories can be planned flexibly rather than pre-enumerated. [15]

Each relocation solves a real problem. Explicit states support execution. Mentalistic formalisms support deliberative agent models. Public commitments support external compliance and flexible planning. Yet each also defines a new boundary. The important research question is therefore not whether context has been removed, but *which predicates were made internal to the formal system and which were left to its environment*.

This perspective also prevents a common historical error. Shared vocabulary does not imply shared machinery. “Promise” in Austin, the Coordinator, Shoham, FIPA, and commitment protocols is not one technical object moving unchanged through time. Sometimes what transfers is a name. Sometimes a causal architecture. Sometimes a formal relation. Sometimes later systems reconstruct a similar problem independently. A defensible lineage must distinguish direct citation, conceptual inheritance, formal resemblance, and our own retrospective comparison.

10 Four semantics for executable institutions

The lineage suggests that institutional programming requires more than a single transition relation. Four semantic layers should be kept distinct even if an implementation later composes them.

1. Event semantics

Event semantics answers: what happened, who acted, when, and in what causal order? It represents utterances, messages, material actions, reports, and observations. Distributed systems make this layer nontrivial because arrival order and causal order can differ. A ledger that totalizes concurrent events is already making a semantic choice.

2. Normative semantics

Normative semantics answers: what public commitments, rights, duties, permissions, prohibitions, or liabilities now hold? This is where the commitment protocols are strongest. A promise or release changes a public relation even after the originating message disappears. Normative state should be inspectable enough to support accountability.

3. Interpretive semantics

Interpretive semantics answers: what did the act count as here, what do its conditions mean, and what would satisfy them? This layer includes the background emphasized by Winograd and Flores and the interactional indeterminacy emphasized by Suchman. It may be actor-indexed, uncertain, or revisable. A request can be formally logged while its practical meaning remains contested.

4. Contestability semantics

Contestability semantics answers: who is authorized to settle an interpretation, challenge a status, waive an obligation, refuse a protocol, or revise a classification? It makes visible the governance otherwise hidden inside context parameters and administrative interfaces. Public semantics without contestability can become merely public coercion.

These layers prevent several category errors. A completion event does not automatically discharge an obligation; normative semantics must say whether it does. A discharge operation does not prove

that material work occurred; event/evidence semantics must connect the ledger to the world. A public commitment does not prove that all parties interpret the commitment identically; interpretive semantics may preserve disagreement. And a resolved dispute does not tell us whether the party who resolved it had legitimate standing; contestability semantics must make that authorization inspectable.

The architecture is not offered as a complete theory of social life. It is a boundary discipline for executable institutions. It says that a system should not claim more semantic closure than it actually implements.

11 Implications for institutional and cultural programming

A programming language for social or cultural processes is often evaluated through familiar technical criteria: syntax, typing, state, transition rules, nondeterminism, and runtime behavior. Those criteria are necessary but insufficient when the program's primitives are categories such as promise, authority, satisfaction, obligation, ritual, status, or norm.

The first additional requirement is *semantic provenance*. If PROMISE is a primitive, the system should specify what theory of promising authorized its semantics and which conditions were deliberately omitted. Shoham's caution that formal mental categories only approximate commonsense concepts is exemplary here. A primitive should be able to say, in effect: this is a model of a social concept under these assumptions, not the concept itself. [11]

The second requirement is *public state without forced unanimity*. Commitment semantics is attractive because it creates inspectable normative relations. Human institutional systems, however, need states richer than OPEN and CLOSED. DISPUTED, WAIVED, MISUNDERSTOOD, IMPOSSIBLE, CONTESTED-AUTHORITY, and UNDER-APPEAL are not error cases to be hidden; they are ordinary institutional states.

The third is *trajectory freedom under semantic constraint*. The strongest answer to finite-state rigidity is not free text attached to a rigid state machine. It is to move invariants from sequence to public normative conditions and permit multiple valid paths. Commitment protocols demonstrate that this is computationally possible. Human-centered systems should investigate the same design space while retaining richer interpretive and contestability semantics. [15]

The fourth is *explicit coupling to the material world*. Winograd and Flores distinguish the conversation coordinating a task from the doing that satisfies it. An institutional runtime should therefore avoid allowing a declaration of completion to silently stand in for completion itself unless the institution explicitly defines that declarative substitution. Some systems will rely on human testimony, others on documents, sensors, inspections, or counterparty acceptance. The evidence rule is part of the semantics.

Finally, the history counsels against a universal language of social action. Austin's contextual conditions, Winograd and Flores's background, Suchman's interactional critique, and Singh's context-sensitive public semantics all point toward typed but revisable locality. A cultural programming system may be most rigorous when it exposes exactly where its semantics stop.

12 Limitations and unresolved territory

Several boundaries of the argument should remain explicit. First, the paper reconstructs a lineage from a finite group of primary and near-primary technical sources. It does not claim that these works form a single self-conscious school. In particular, the conceptual convergence between Suchman's CSCW critique and Singh's agent-communication critique is an interpretive comparison, not a documented influence claim. The same is true of the relation between Winograd/Flores's language of commitment and later commitment-based agent protocols unless direct citation evidence is recovered.

Second, the paper has emphasized semantic architecture more than empirical deployment. The Coordinator's finite-state machinery and workflow categories are documented, but their actual use across organizations requires a different evidentiary base: manuals, source code, field studies, user reports, screenshots, organizational policies, and records of workaround or refusal. Such evidence could substantially qualify the claim that formal categories constrained action in practice.

Third, public commitment semantics solves a specific problem - normative testability across autonomous participants - without solving the whole problem of institutional power. A publicly visible commitment can still be imposed under unequal conditions. Authority, ownership, enforcement, and sanction remain underdeveloped in the formal lineage traced here.

Fourth, the four-layer architecture is a synthesis proposed by this paper. It has not been demonstrated to be minimal or complete. A mature institutional programming language may need additional semantics for evidence, identity, collective actors, temporal uncertainty, resources, coercion, and appeal. The architecture should therefore be treated as a testable design decomposition rather than a finished ontology.

The strongest next research move is a set of adversarial implementations. Rebuild a request-promise-completion protocol four ways: as a finite-state workflow; as a mentalistic FIPA-style protocol; as a public commitment protocol; and as a contestable commitment protocol with actor-indexed interpretation and authority challenges. Feed all four the same hard cases: unauthorized promises, ambiguous requests, disputed satisfaction, delayed or concurrent messages, waived obligations, deceptive agents, and refusal of the protocol itself. The differences would make the migration of semantic burden empirically inspectable.

13 Conclusion

Speech-act theory did become executable, but not by a single translation from philosophy into code. The history is a sequence of relocations. Austin distributes performative success across convention, actor, circumstance, intention, and later conduct. Winograd and Flores combine speech acts with a hermeneutic account of background while making commitment central to coordination. The Coordinator and ActionWorkflow compile selected parts of that structure into roles, states, permitted moves, and persistent incompletions. Agent-oriented programming relocates semantic burden into formal mental state; FIPA standardizes that strategy; Singh moves normative meaning toward the public social layer; and commitment protocols finally make obligations into explicit, executable relations whose trajectories can remain flexible. [1, 3, 6, 11, 13, 14, 15]

The deepest lesson is not that formalization inevitably destroys context. It is that formalization decides where context will live. A context can become a guard, a role, an internal belief, a public

convention, a commitment fluent, or an unresolved dispute. Those choices determine not only what software can execute but what an institution can see, test, challenge, and remember.

Executable social order therefore needs a more demanding standard than syntactic correctness or successful state transition. A serious institutional semantics must say what happened, what obligations now hold, how their meaning is interpreted, and who can contest the answer. The protocol becomes intellectually defensible when those layers are distinguished rather than silently collapsed. The result is not less computation. It is computation that knows where its authority ends.

Appendix A — Assembly instrument

The exact visible assembly prompt used for this publication is preserved as `SLIPCASE_FINAL_PROMPT.txt` and `_PROMPTS/001__SLIPCASE_ANCIENT_MASTER_15.0.txt`. Raw chat-transport bytes are unavailable to the build environment; the package therefore records the prompt as a transcription of the visible user turn and does not claim platform-level byte identity.

Appendix B — Making history

The publication distinguishes provided, preserved, retrieved, derived, verified, and unverified material in `000__MAKING_HISTORY.txt`. The 80 inherited numbered research payloads were copied from the preceding portable field. Thirty-two new research objects were produced from the lineage inquiry. The terms “migration of semantic burden” and the four-layer semantic architecture are synthetic contributions of this paper, not terminology attributed to the cited authors.

Appendix C — Replication path

Open `index.html` for the portable research capsule, then read `000__RETURN_PATH.txt`, `000__OPEN_EDGES.txt`, and `felicity-to-protocol__SOURCE_MAP.txt`. Machine-readable state is under `_SLIPCASE/`. Run `python3_SLIPCASE/rebuild.py` to verify immutable card hashes and rebuild core indexes.

References

- [1] J. L. Austin. *How to Do Things with Words*. Oxford: Clarendon Press, 1962.
- [2] John R. Searle. *Speech Acts: An Essay in the Philosophy of Language*. Cambridge University Press, 1969.
- [3] Terry Winograd and Fernando Flores. *Understanding Computers and Cognition: A New Foundation for Design*. Norwood, NJ: Ablex, 1986.
- [4] Terry Winograd. “A Language/Action Perspective on the Design of Cooperative Work.” *Human-Computer Interaction* 3(1), 1987: 3–30.

- [5] Fernando Flores, Michael Graves, Brad Hartfield, and Terry Winograd. “Computer Systems and the Design of Organizational Interaction.” *ACM Transactions on Office Information Systems* 6(2), 1988: 153–172.
- [6] Raul Medina-Mora, Terry Winograd, Rodrigo Flores, and Fernando Flores. “The Action Workflow Approach to Workflow Management Technology.” In *Proceedings of CSCW 1992*, 281–288. ACM, 1992.
- [7] Carlos F. Flores et al. *Method and Apparatus for Structuring and Managing Human Communications by Explicitly Defining the Types of Communications Permitted Between Participants*. US Patent 5,208,748, 1993.
- [8] Lucy Suchman. “Do Categories Have Politics? The Language/Action Perspective Reconsidered.” *Computer Supported Cooperative Work* 2, 1993: 177–190.
- [9] Terry Winograd. “Categories, Disciplines, and Social Coordination.” *Computer Supported Cooperative Work* 2, 1994: 191–197.
- [10] Lucy Suchman. “Speech Acts and Voices: Response to Winograd et al.” *Computer Supported Cooperative Work* 3, 1995: 85–95.
- [11] Yoav Shoham. “Agent-oriented programming.” *Artificial Intelligence* 60(1), 1993: 51–92.
- [12] Ira A. Smith and Philip R. Cohen. “Toward a Semantics for an Agent Communications Language Based on Speech-Acts.” In *Proceedings of AAAI-96*, 24–31. AAAI Press, 1996.
- [13] Foundation for Intelligent Physical Agents. *FIPA Communicative Act Library Specification*, SC00037J, 2002.
- [14] Munindar P. Singh. “Agent Communication Languages: Rethinking the Principles.” *Computer* 31(12), 1998: 40–47.
- [15] Pinar Yolum and Munindar P. Singh. “Flexible Protocol Specification and Execution: Applying Event Calculus Planning Using Commitments.” In *Proceedings of AAMAS 2002*, 527–534. ACM, 2002.